CSE247 Data Structures (Fall'25)

Midterm Examination

Max Marks: 50 *Time Allowed:* 2 hours

Instructions:

- Answer the questions in the spaces provided on the question sheets.
- Extra sheets may be used for rough work, but make sure to write your name and ERP on them. Answers on extra sheets will not be graded.
- Understanding the questions is part of the exam, please do not ask for clarifications during the exam.
- While describing algorithms, you may use pseudocode or a mix of pseudocode and C++ code.

Name: _____ **ERP:** _____

Question:	Union/Find	Priority Queues	Binary search trees	Balanced search trees	Total
Marks:	10	10	19	11	50
Score:					

Question 1: Union/Find ..... 10 marks

(a) [3 marks] Consider the following parent-link representation of a weighted quick union (link-by-size) data structure.

	0	1	2	3	4	5	6	7	8	9
`parent[]`	9	3	7	?	3	8	9	8	8	9

Which of the following values could be `parent[3]`? Fill in all checkboxes that apply.

0	1	2	3	4	5	6	7	8	9
◯	◯	◯	✓	◯	◯	◯	◯	✓	✓

(b) [3 marks] A fellow student suggests adding an extra array, called `root`, in the *quick-union* implementation to store the root of each element. He argues that by maintaining this `root` array, the time complexity of the `find` operation becomes constant, and therefore the `union` operation also becomes constant-time.

Do you agree or disagree with this claim? Mark one checkbox: ◯ Agree ✓ Disagree

Give a reason for your answer.

Solution: Because whenever two trees are merged during a union, the roots of potentially many elements change. Updating the root array for all affected elements would require traversing an entire tree, making the operation $O(n)$, not $O(1)$.

(c) [4 marks] The typical API of a quick-union data structure for n elements includes the methods `initialize(n)`, `find(p)`, and `unify(p,q)`. Suppose we want to extend this API with a new method:

```
void unify_batch(const std::vector<int>& batch);
```

where batch contains m distinct element indices ($m \leq n$), and *each element in the batch is initially disconnected from all others*. The purpose of `unify_batch` is to connect all elements in the batch so that they belong to the same component. The implementation should:

- run in $O(m)$ time, and
- ensure that, for any of the m elements, a subsequent call to `find()` runs in $O(1)$ time.

In the space below, describe in concise English (and optional pseudocode) the logical steps needed to implement the `unify_batch` method efficiently.

Solution: Choose one element (say `batch[0]`) as the root of the new component. For each remaining element `x` in the batch: Set `parent[x] = root`. Update auxiliary arrays (like `size[]`) accordingly, such as set `size[root] = m`.

Note: the for loop runs in $O(m)$ and the height of the new tree remains 1, so `find(x)` for any `x` in the batch is $O(1)$.

Question 2: Priority Queues ..... 10 marks

(a) [5 marks] Consider the array `A[]`

-	29	18	10	15	20	9	5	13	2	4
0	1	2	3	4	5	6	7	8	9	10

i. Does `A` satisfy the max-heap property? If not, fix it by swapping two elements.

Solution: No, to make it a max-heap swap the values 18 and 20.

ii. Using array `A` (possibly corrected), illustrate the execution of the DEL-MAX algorithm, which extracts the maximum element and then rearranges the array to satisfy the max-heap property. For each iteration or recursion of the algorithm, write the content of the array `A`.

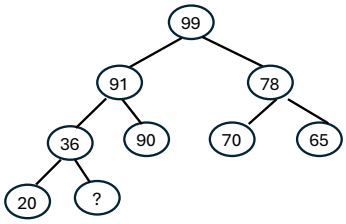
Solution:

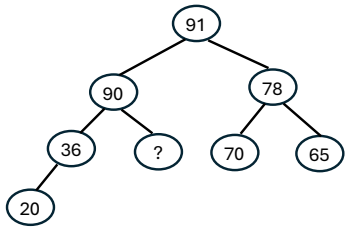
```
A = [29, 20, 10, 15, 18, 9, 5, 13, 2, 4]
A = [ 4, 20, 10, 15, 18, 9, 5, 13, 2, 29]
A = [20,  4, 10, 15, 18, 9, 5, 13, 2]
A = [20, 18, 10, 15,  4, 9, 5, 13, 2]
```

(b) [3 marks] A set of keys is stored in a max-heap `H` and in a binary search tree `T`. Which data structure offers the most efficient algorithm to output all the keys in descending order? Or are the two equivalents? show their time complexities.

> **Solution:** A max-heap requires $O(n \log n)$ time to output all keys in descending order because each DELETE-MAX operation takes $O(\log n)$ time (to remove the root and re-heapify), and this must be done for all n elements. In contrast, a BST can output all keys in descending order in $O(n)$ time using a reverse in-order traversal, since each node is visited exactly once. Therefore, the BST is more efficient than the max-heap for producing all keys in descending order.

(c) [2 marks] Consider a max-heap on the left (with no duplicate keys) which transforms to max-heap on the right after one del-max operation. Indicate what could be the values of the question mark.



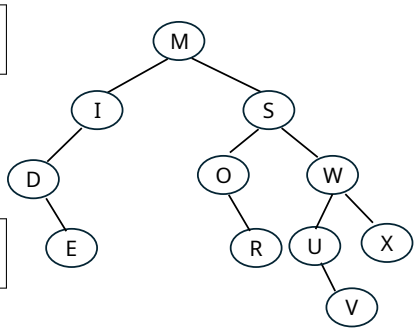


Fill in all checkboxes that apply.

☐ 50	☑ 27	☐ 55	☐ 38	☐ 89	☑ 30	☐ 47

Question 3: Binary search trees ..... 19 marks

(a) [4+3 marks] .
Consider the tree on the right:



i. List the keys seen in a *post-order* traversal of the tree.

> **Solution:** `E D I R O V U X W S M`

ii. List the keys seen in a *pre-order* traversal of the tree.

> **Solution:** `M I D E S O R W U V X`

iii. List the keys seen in an *in-order* traversal of the tree.

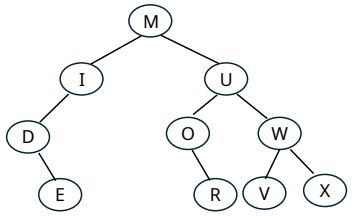
> **Solution:** `D E I M O R S U V W X`

iv. In the given binary search tree, with keys corresponding to the letters shown, which of these keys are compared to '`P`' when one attempts to retrieve the value associated with '`P`'? Write these keys in the order they are compared.

Solution: `M S O R`

v. Show the tree state after the *Hibbard's* deletion (using successors) of the node corresponding to key '`S`'.

Solution:



(b) [3 marks] .

Given the definition of the binary search tree method (pseudocode) on the right, answer the following questions.

i. What does the mystery method do on a given binary search tree?

ii. What does the variable '`x`' keep track of?

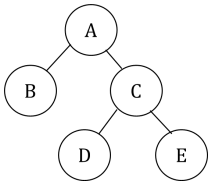
iii. What does the variable '`y`' keep track of?

```
1  mystery (T,  z)
2      y = NULL
3      x = T->root
4      while x != NULL
5          y = x
6          if z->key < x->key
7              x = x->left
8          else x = x->right
9      if y == NULL
10         T->root = z
11     else if z->key < y->key
12         y->left = z
13     else y->right = z
```

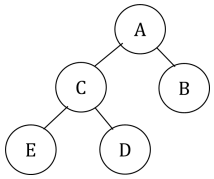
Solution: The algorithm does perform BST insertion, but with the convention that duplicates go to the right.

- The method inserts node `z` into the binary search tree `T`. If a node with the same key already exists, the new node is placed in the right subtree of that node.
- `x` keeps track of the current node being inspected while searching for the insertion point.
- `y` keeps track of the parent of `x`; after the loop, it is the parent of the new node `z`.

(c) [5 marks] Write a recursive C++ function **`void flip(Node* root)`** to flip the original tree by updating the original keys and values so that the tree is now the mirror image of its original value. *Do not create a new tree, just update the original tree.*



Before



After

Solution:

```
void flip(Node* root) {
    if (root == nullptr)
        return;

    // Recursively flip left and right subtrees
    flip(root->left);
    flip(root->right);

    // Swap left and right child pointers
    Node* temp = root->left;
    root->left = root->right;
    root->right = temp;
}
```

(d) [4 marks] Answer the following precisely (no tilde or big-O notation). Justify your answers.

i. What is the minimum number of nodes in a complete binary tree with depth 3?

Solution: A *complete binary tree* has all levels filled except possibly the last, and the last level filled from left to right.
The minimum number of nodes occurs when the last level has only one node.
Number of nodes up to level 2 (fully filled): $1 + 2 + 4 = 7$.
Adding 1 node on level 3, so $7 + 1 = 8$ is the answer.

ii. What is the height of the "shortest" binary search tree that has 403 nodes?

Solution: The *shortest* BST (i.e., minimum height) is a perfectly balanced binary tree.
For a perfectly balanced tree with height h:

$$n_{\max} = 2^{h+1} - 1$$

We need the smallest h such that $2^{h+1} - 1 \geq 403$

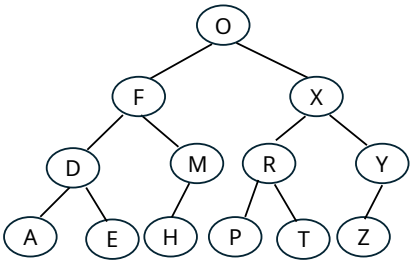
$$2^8 - 1 = 255 < 403, \quad 2^9 - 1 = 511 \geq 403$$

So $h = 8$.

iii. What is the height of the "tallest" binary search tree that has 403 nodes?

Solution: The tallest BST is the most unbalanced one (like a linked list).
If all nodes form a chain, the height equals number of edges $= n - 1$. So $h = 403 - 1 = 402$.

iv. Is the following binary search tree in symmetric order?

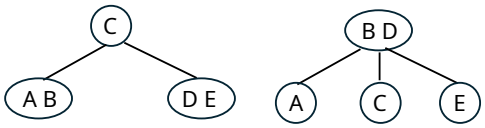


Solution: No, `Z` cannot be left child of `Y` as `Z` > `Y`.

Question 4: Balanced search trees ..... 11 marks

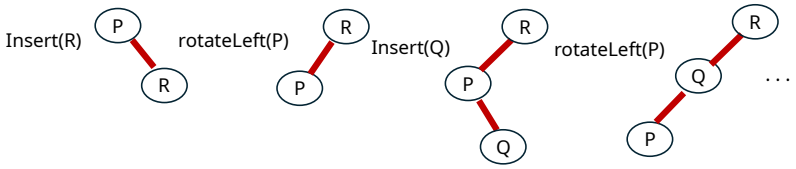
(a) [3 marks] Dr. Amongus claims that a 2-3-tree containing a given set of entries always has the same structure, no matter the order of insertion. Demonstrate that this claim is false.

Solution: Here is a *counter example*. To get left tree, insert in order: `A C E B D`. While to get the right tree, the order should be `A B C D E`.



(b) [3 marks] Dr. Amongus claims that when inserting keys into an left-leaning red-black BST, *no sequence of insertions ever requires two consecutive left rotations.* Is this claim true or false? Provide a counterexample or reasoning.

Solution: False, as shown in the following example.



Inserting `R` into single-node tree `P` cause a left-rotation. A subsequent `insert(Q)` will cause

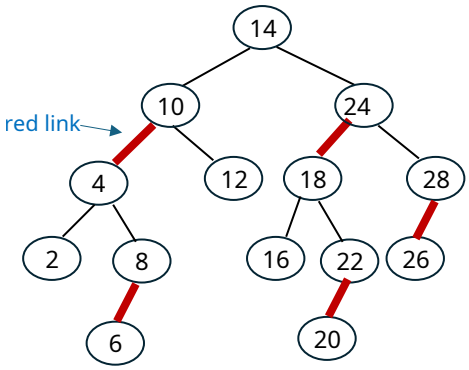
another left-rotation.

However, note that two left rotations can't happen consecutively during a single LLRB insertion. After inserting the new node with a red-link, the standard LLRB restore-steps are executed in this exact order on every node on the path from newly inserted node to root:

1. `if (isRed(h->right)&& !isRed(h->left))h = rotateLeft(h);`
2. `if (isRed(h->left)&& isRed(h->left->left))h = rotateRight(h);`
3. `if (isRed(h->left)&& isRed(h->right))flipColors(h);`

After a left rotation performed in step (1), either the LLRB color-invariants are restored, or it is immediately followed by right-rotation, and a color-flip. So two left-rotation cannot happen consecutively in a single insertion.

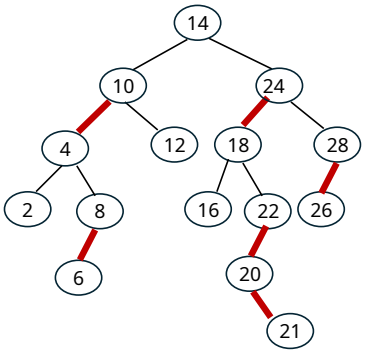
(c) [5 marks] Consider the following left-leaning red-black BST:



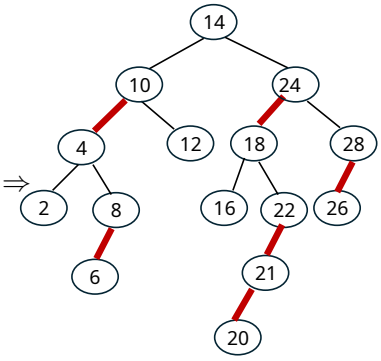
Insert the key 21 into the tree. List the intermediate trees after each step (rotations, color flips etc) of the insertion process.

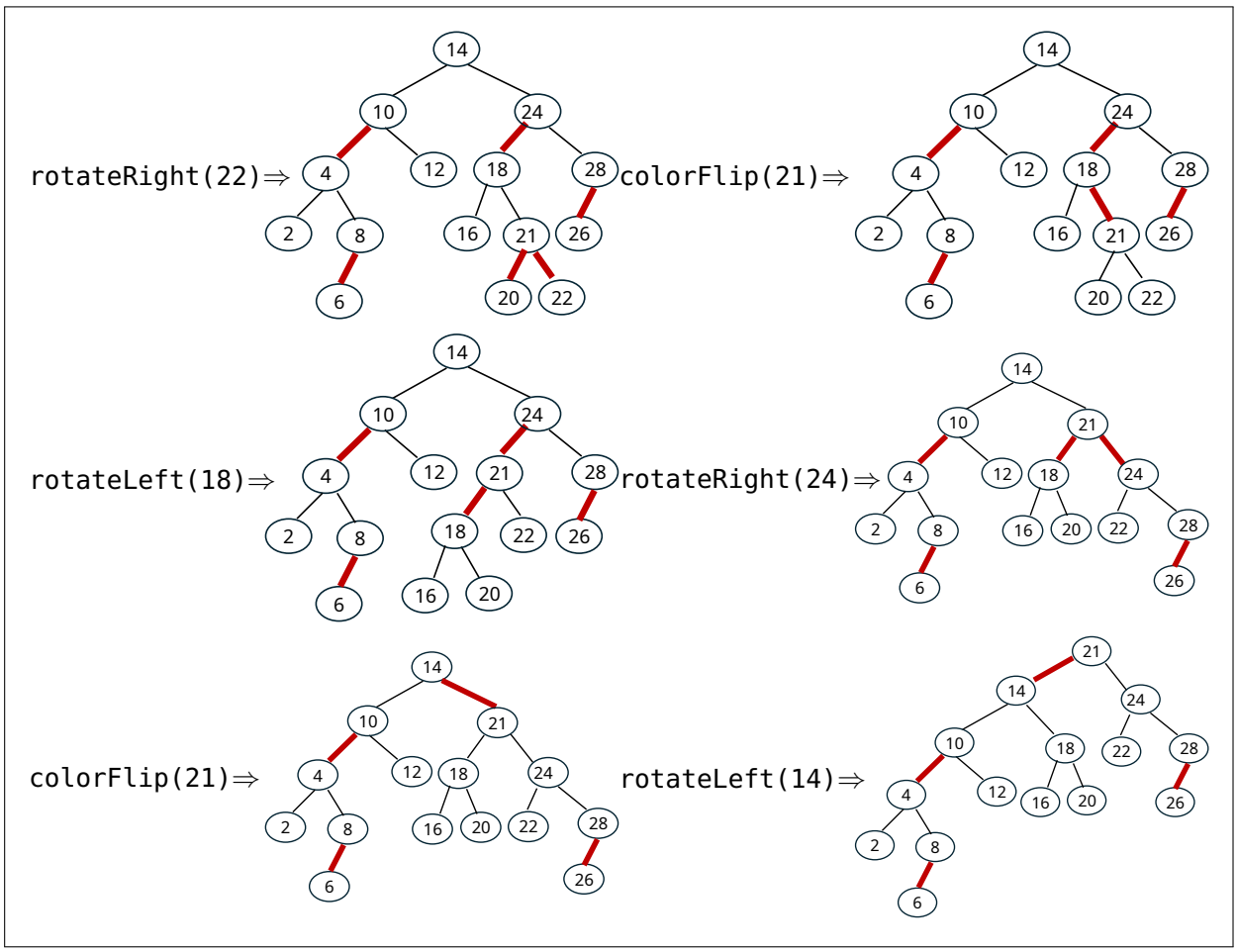
Solution:

`insert(21)`⇒



`rotateLeft(20)`⇒




rotateRight(22)⇒
colorFlip(21)⇒
rotateLeft(18)⇒
rotateRight(24)⇒
colorFlip(21)⇒
rotateLeft(14)⇒